

www.3aayaam.in

Knowing Spark

Up Close 2

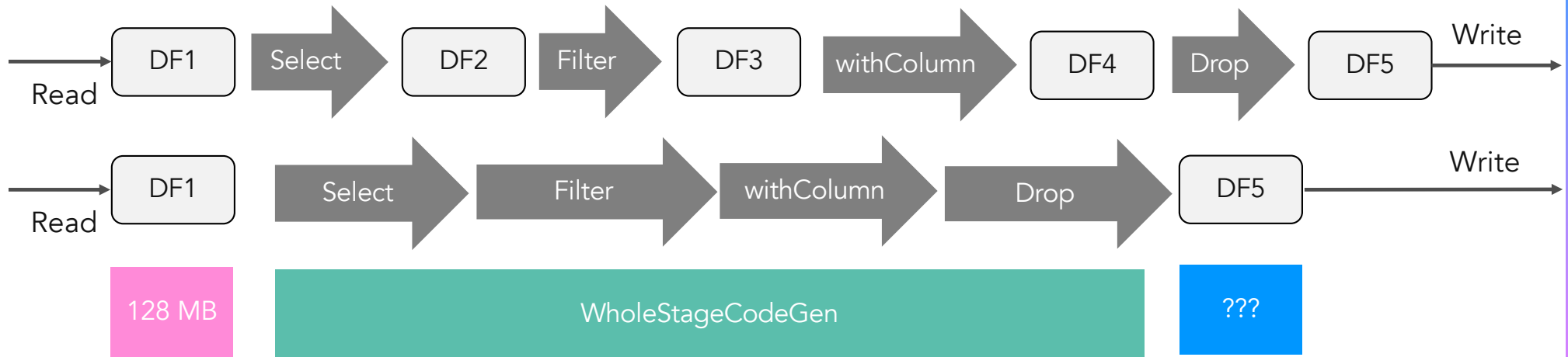
Agenda

1. Memory & CPU Usage
2. WholeStageCodegen, MapPartitions, Skipped Stages
3. Spark Memory Management
4. Off-heap Memory
5. Memory Spill
6. Data Skewness
7. Salting
8. Execution Path
9. Adaptive Query Execution
10. Dynamic Resource Allocation

Memory & CPU Footprint

Will new DataFrames keep
on occupying memory ??

DAG Memory Usage



- Spark does not store all DataFrame in memory.
- Input DataFrame, Output DataFrame and Staged DataFrames use memory while reading and writing.

How much Memory Needed?

128 MB

WholeStageCodeGen

???

- Memory usage 1 – Input DataFrame, while reading input source (file, databases, stream sources, data warehouses etc.).
- Memory usage 2 – Across stages.
- Memory usage 3 – Output DataFrame, while writing to output targets (file, database, stream targets, dwh etc.).

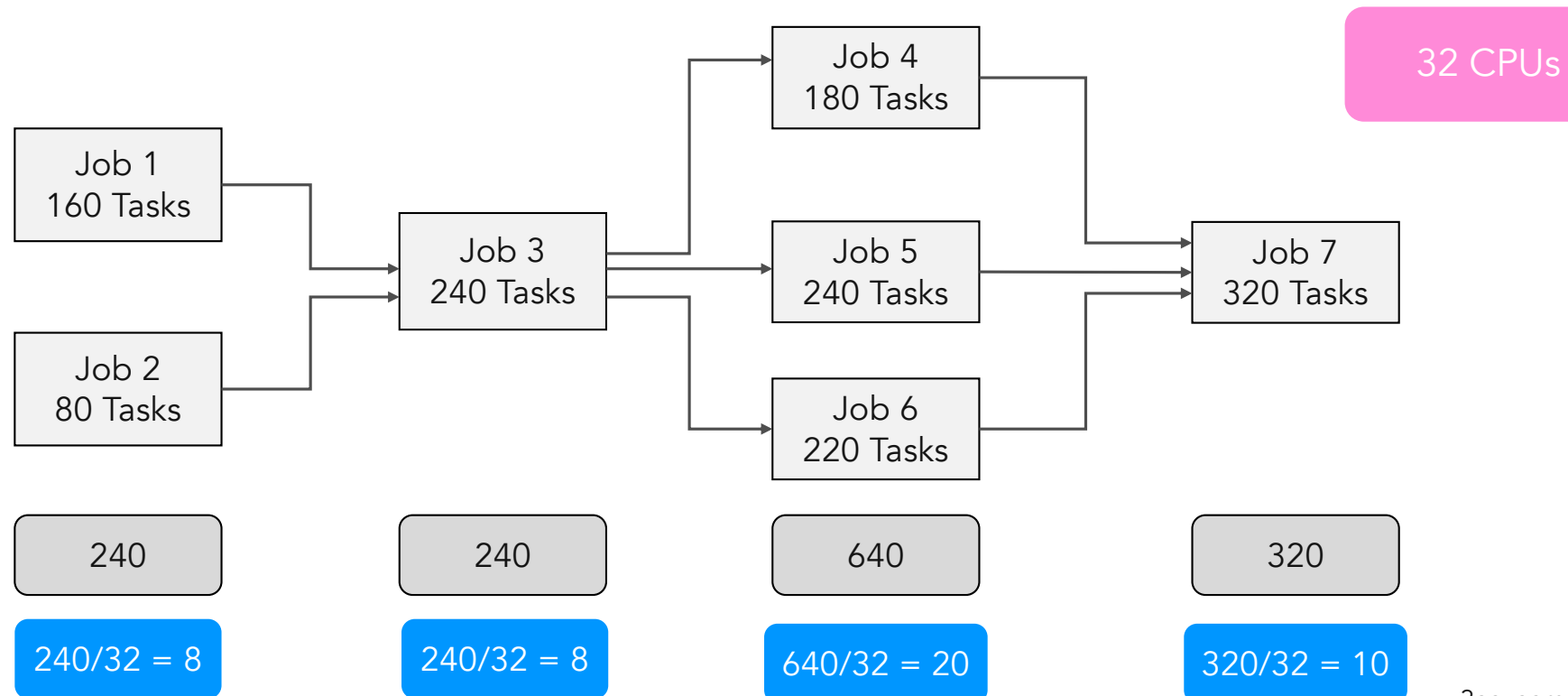
Memory for input DataFrame =
 $128 * \text{Total Concurrent Tasks that can run}$

Memory for output DataFrame =
Have to be found using Spark UI

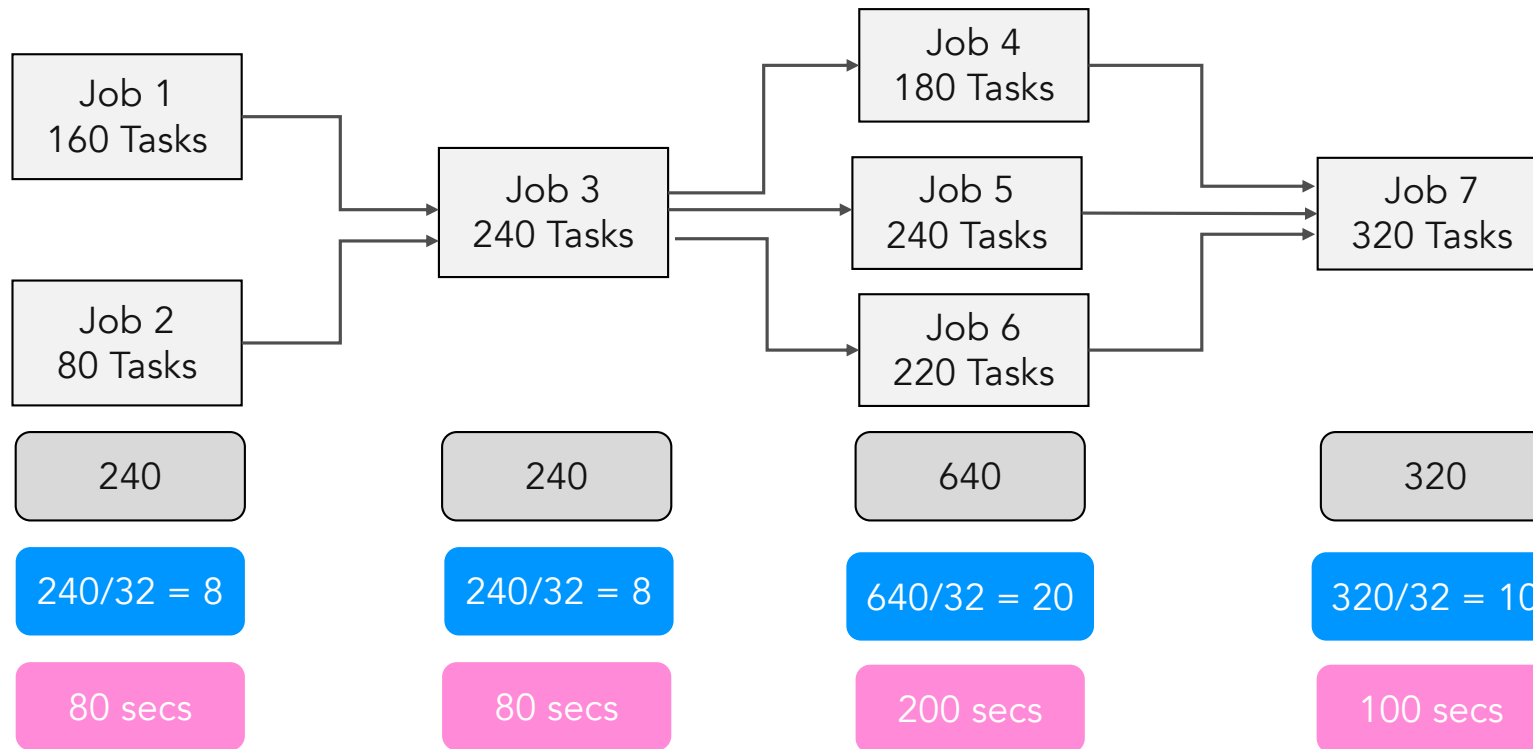
Total Memory Usage = Memory for i/p DataFrame + Memory for o/p DataFrame

How many CPUs Needed?

- Total CPU requirement = Sum of all tasks of concurrent jobs (Maximum Parallelism)



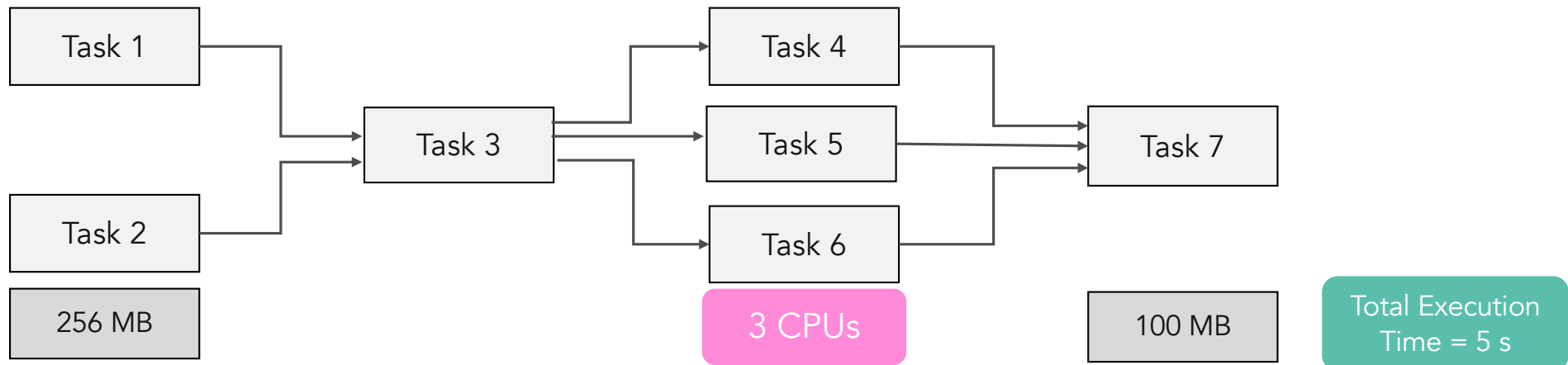
How many CPUs Needed?



Total Execution
Time = 460 s

Decide Spark Cluster Size

- Size the production cluster using UAT.
- Same dataset size is not available.
- Complex applications should complete within 5 mins (300 s) for input data size of 500GB.



- $256 \text{ MB with } 3 \text{ CPUs} = 5\text{s} \rightarrow 256 \text{ MB with } 1 \text{ CPU} = 15\text{s} \rightarrow 1 \text{ MB with } 1 \text{ CPU} = (15/256) \text{ s}$
- $500 \text{ GB with } 1 \text{ CPU} = (15 * 512000/256) = 30,000 \text{ s} \rightarrow 100 \text{ CPUs}$

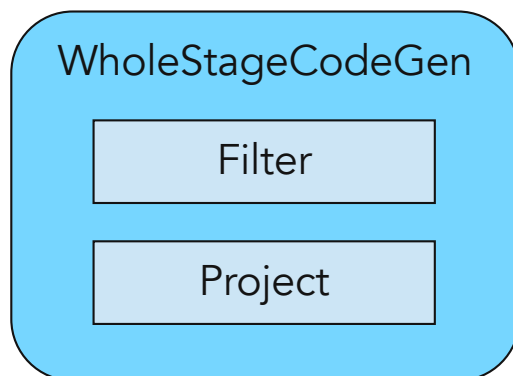
Decide Cluster Size - Exercise

- Given the following requirement and experimentation, what should be the size of the production cluster (CPU and Memory)
 - With 16 CPUs and 2 GB of input data the total execution time was 38s.
 - Need to complete a Spark application in 10 mins for input data size of 1.5 TB
- Spark UI – Find the input and output DataFrame size
- **spark.executor.memory** – Amount of memory per executor.
- **spark.executor.cores** – No. of CPUs per Executor.

Exploring Spark UI More...

DAG Visualization – Spark UI

- WholeStageCodeGen



- Map
- MapPartitions
- MapPartitionsInternal
- MapPartitionsWithIndex

Map

For NT - Spark iterates through each row and applies the necessary function to each row or column.
select, filter, withColumn.

MapPartitions

For NT - Spark applies the necessary function to each partition rather than row-by-row.
select, filter, withColumn

MapPartitionsInternal

For NT - Spark's "internal" implementation of MapPartitions. Spark does not have API for this.

MapPartitionsWithIndex

For NT - MapPartitions with Index for the partitions.

DAG Visualization – Spark UI

- Exchange – Shuffle Process (ShuffleWriter + ShuffleReader).
- LocalTableScan – Reading from memory of executors.
- BroadcastExchange – Implementation of Join Strategy.
- InMemoryTableScan
- Skipped Stages (Greyed out) – Stages that are not executed again because it was already executed earlier and the output is available for use.

Spark Memory Management

Change Configuration at Run Time

- Change driver/executor memory.
- Change shuffle partitions.
- Enable/disable certain spark config parameters.
- Can be done at:
 - Application Level – using SparkConf.
 - While submitting job – using spark-submit.

```
from pyspark import SparkConf

conf = SparkConf()
conf.set("spark.executor.memory", "6g")
conf.set("spark.memory.offHeap.size", "4g")

spark = SparkSession.builder.config(conf=conf)...
```

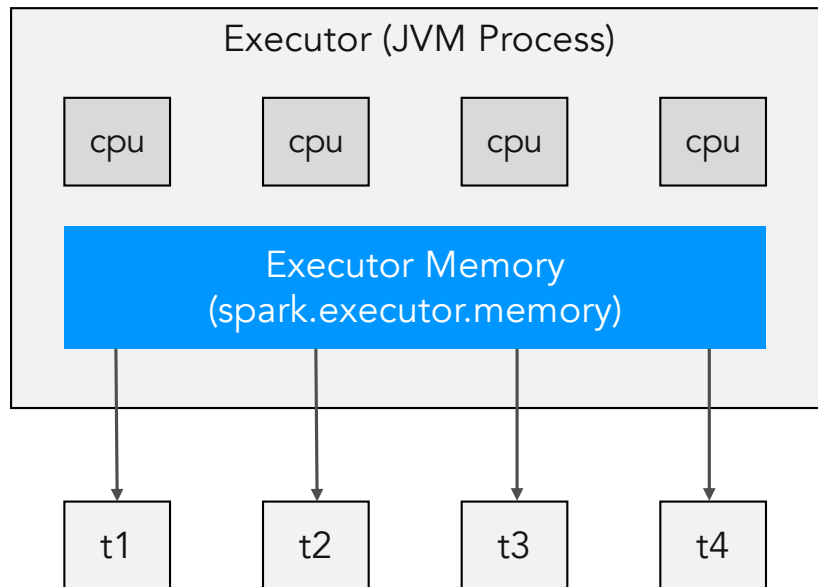
```
spark-submit --driver-memory 4g --executor-memory 6g --executor-cores 1 --num-executors 1
--conf spark.memory.fraction=0.9 pyspark_code.py
```

No. of Executors - Formula

No. of Executors =

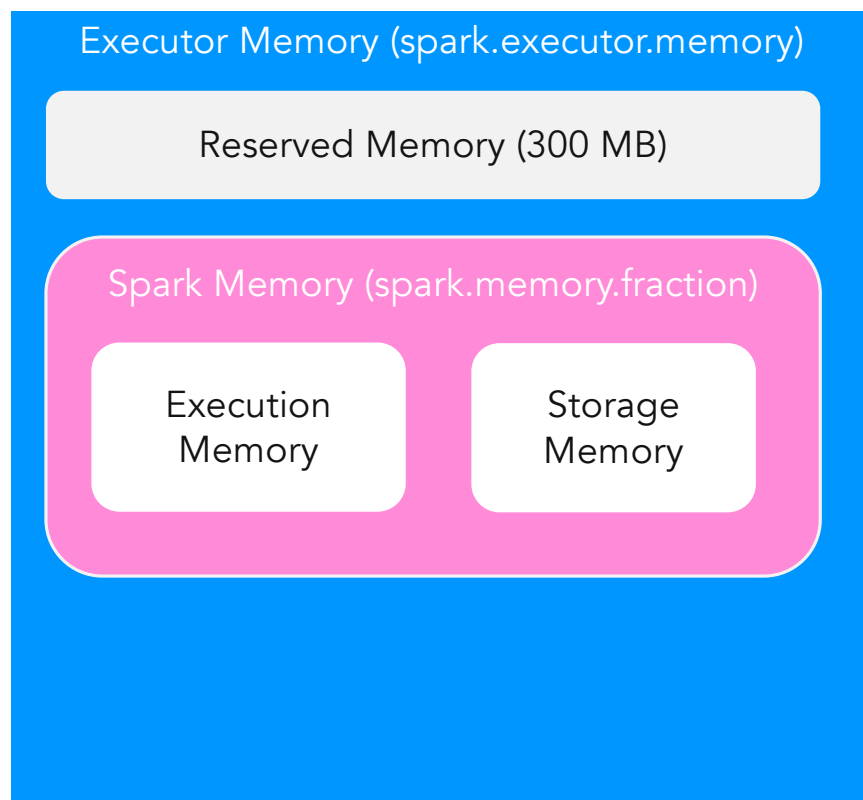
MAX((Total CPUs in all Worker Nodes / No. of CPU per Executor),
(Total Memory in all Worker Nodes / Amount of Memory per Executor))

Spark Executor Memory & Cores



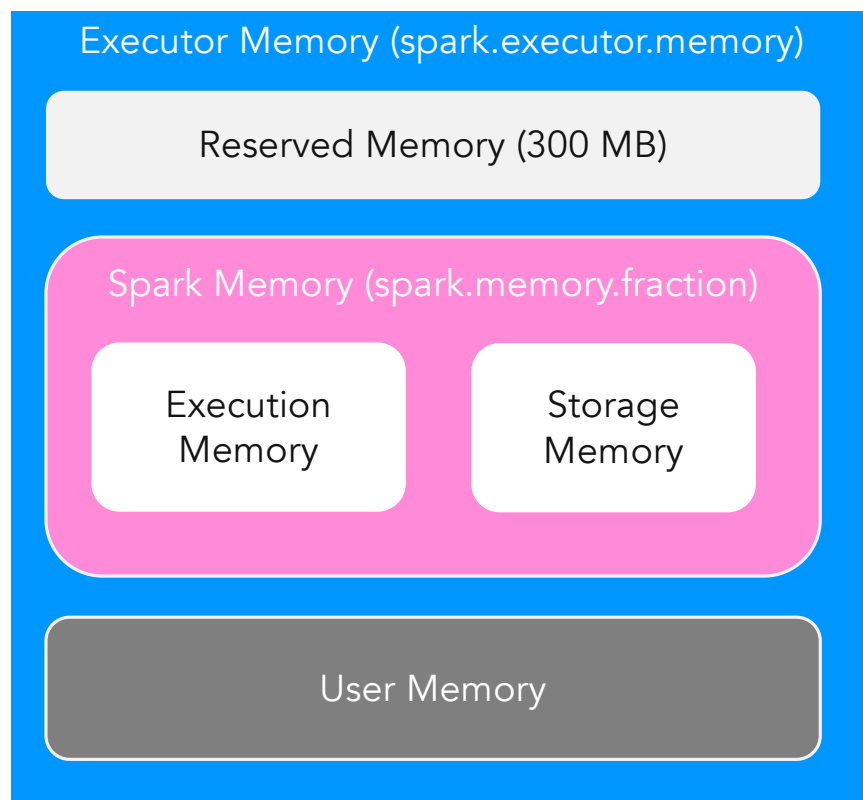
- `spark.executor.memory` & `spark.executor.cores`
- Memory allocation is at executor level.
- Can be modified from Spark Application only using `SparkConf`.
- Executor Memory (`spark.executor.memory`) sets the JVM heap size for an executor.

Spark On-heap Memory



- Executor Memory has 3 regions
- Reserved memory
 - Used for Spark's internal objects.
 - Non-modifiable with a value of 300 MB.
 - Buffer to avoid OOM errors for tasks.
- Spark memory (spark.memory.fraction = 0.6)
 - Execution Memory – sort, join, agg, shuffle etc.
 - "Peak Execution Memory OnHeap/OffHeap"
 - Storage Memory (spark.memory.storageFraction) – Used for caching dataframes. cache(), persist().
 - "Peak Storage Memory OnHeap/OffHeap"

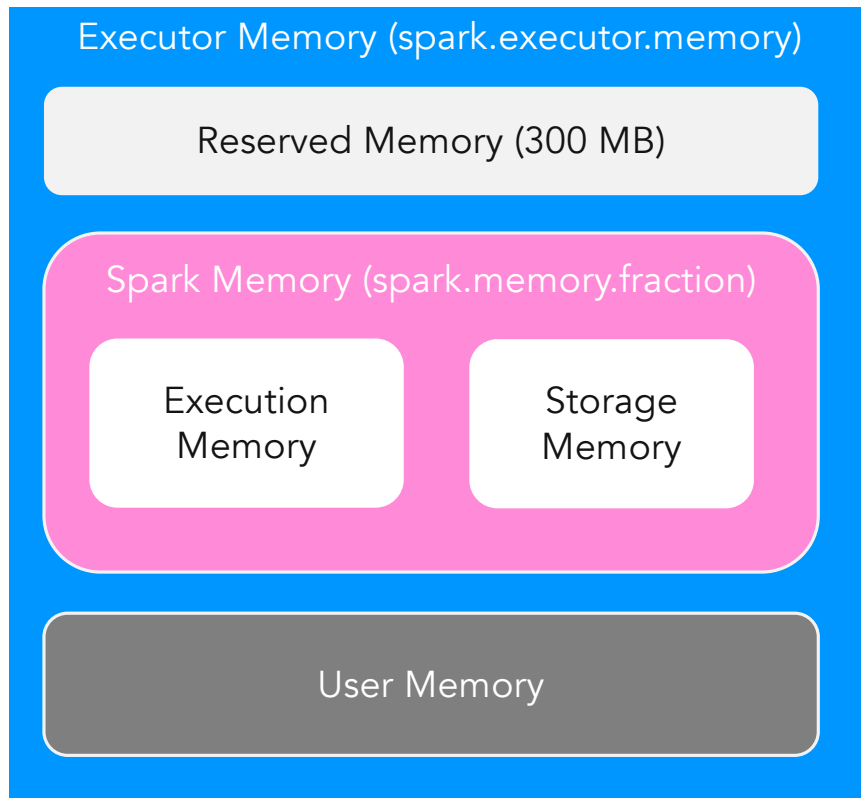
Spark On-heap Memory



- User Memory – For user defined variables, functions, UDFs etc.
- Actual value of allocated memory:
 - `spark.executor.memory` = 2 GB.
 - Reserved Memory = 300 MB. Remaining Memory = 1.7 GB.
 - Spark Memory (0.6) = 1.7 GB * 0.6 = 1.02 GB
 - `spark.memory.storageFraction` = 0.3
 - Execution memory = 1.02 * 0.7 = 0.714 GB
 - User Memory = 1.7 – 1.02 = 0.68 GB
- Execution and Storage memory can use each other's memory when needed.

Spark UI → Stages tab

Memory Spilling



- Execution & Storage memory can spill to disk if enough RAM is not available. When value of `spark.memory.fraction` is not enough.
- Execution Memory - Sort, Join, Shuffle, Aggregation.
- Storage Memory – `cache()`, `persist()`.
- When these operations needs more memory and is not available, then it can be spilled.

Memory Spilling

- 2 types of spill in Spark:
- Memory Spill:
 - Execution memory is using some or all of Storage memory, if available.
 - Vice-versa.
 - Off-heap memory
- Disk Spill: Data is spilled to disk.
- Optimize spilling: optimize configuration
 - **`spark.executor.memory`**
 - **`spark.memory.fraction`**
 - **`spark.sql.shuffle.partitions`**

Spark Memory (`spark.memory.fraction`)

Execution Memory

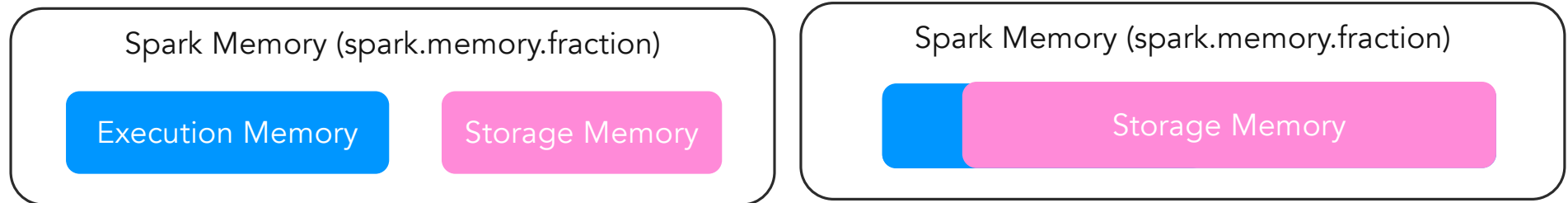
Storage Memory

Spark Memory (`spark.memory.fraction`)

Storage Memory

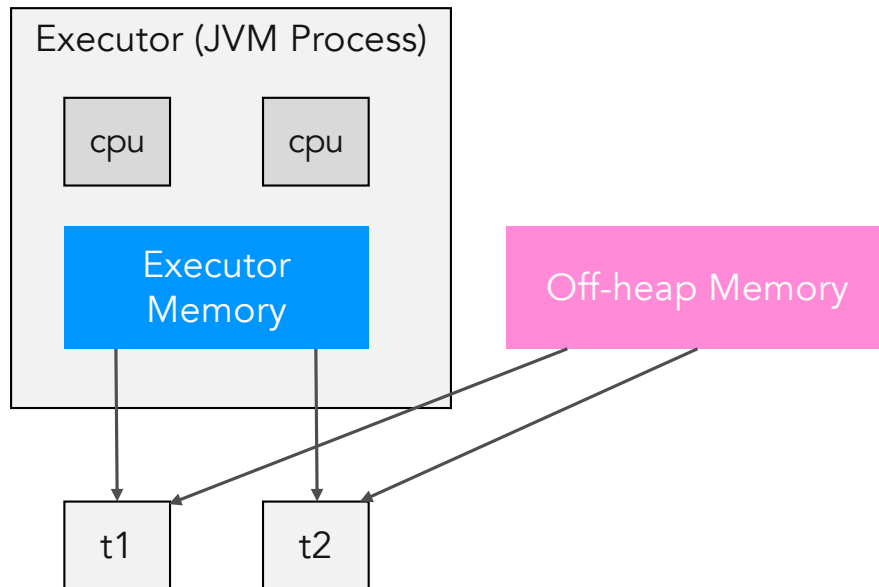
Identification using Spark UI.
SQL/Dataframe & Stages tab.

Unified Memory Management



- Unified memory pool.
- Pertains to Spark Memory (spark.memory.fraction) only.
- Dynamic memory sharing between Execution & Storage memory.
- **spark.memory.fraction** & **spark.memory.storageFraction**.

Off-heap Memory



- Outside of Executor JVM memory.
- Used by – i) Execution Memory & ii) Storage Memory.
- Configuration:
 - `spark.memory.offHeap.enabled`
 - `spark.memory.offHeap.size`
- More RAM access.
- Garbage Collection – Temp objects created by transformations, shuffle process, cached DFs, internal spark objects.
- Lesser Garbage Collection overhead.
- Cleared by OS automatically.

Cache & Persist

- Used to save a dataframe in memory or disk.
- `from pyspark.storagelevel import StorageLevel`
- `persist(StorageLevel.<options>)`
 - MEMORY_ONLY & MEMORY_ONLY_SER – When there's enough memory. Data is never spilled to disk. Minimum CPU overhead.
 - DISK_ONLY – Too large dataset. Disk IO involved.
 - MEMORY_AND_DISK & MEMORY_AND_DISK_SER – Balance between memory and disk. CPU overhead would be there for serialization and deserialization.
 - OFF_HEAP – Data is stored off-heap in serialized form.
- `cache()`
- Remove from memory: `spark.catalog.clearCache()` & `df.unpersist()`

JVM Garbage Collection (GC)

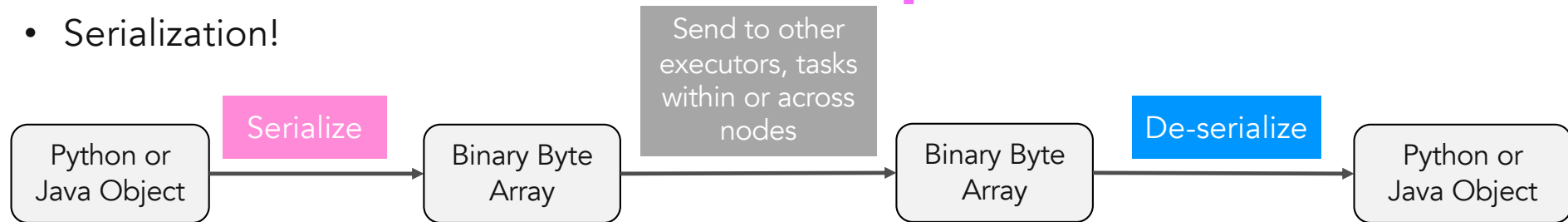
- GC is an automatic memory management process.
- Purpose of GC is to remove objects in-memory that are not getting used.
- Minor GC (Young generation memory area) & Major GC (Old generation memory area).
- GC performance is very critical to Spark as it is a memory-oriented processing framework.
- Dataframe in memory using `persist()` can cause GC. Use `unpersist()` to remove these objects when usage is no more needed.
- Monitor using “GC Time” in Spark UI. If it is more than 10-20% of total task time, then it should be flag for concern.
 - Longer task execution time.
 - Java heap space memory errors.
 - Can stop executors to send communication to driver.

Tune JVM GC

- Tuning JVM issues:
 - `spark.executor.memory` & `spark.driver.memory`
 - Don't create too many objects. Use chain transformation as much as possible for large code bases.
 - Use Parquet / Avro files as much as possible and filter data at the earliest.
 - Use `MEMORY_AND_DISK_SER` to reduce memory usage by cached dataframes.
 - Kryo serialization.
 - `unpersist()` or `spark.catalog.clearCache()` dataframes when no longer needed.

Serialization in Spark

- Serialization!



- When Spark serializes objects?
 - Shuffling (join, aggregation, groupBy, sort, distinct ...)
 - Persist or Cache using MEMORY_ONLY_SER and MEMORY_AND_DISK_SER
 - CPU and Memory overhead
- Spark Serialization Algorithms:
 - Java Serialization – java.io.Serializable (slow, higher memory usage)
 - Kryo Serialization – org.apache.spark.serializer.KryoSerializer (fast, less memory usage, less GC overhead)
 - `conf.set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")`

Summary

www.3aayaam.in

- Dataframe & Memory
- Memory & CPU Requirement
- Calculate Spark Cluster Size
- Spark UI
- How to change configuration at Run Time
- On-heap or Executor Memory
- Off-heap Memory
- Execution & Storage Memory
- Unified Memory Management
- Memory Spill
- Spark JVM Garbage Collection
- Kryo Serialization

Project 3 (Flight Transportation Data)

departure_time
arrival_time
flight_id
aircraft_id
itinerary_no
ticket_no
flight_cost
origin_airport
destination_airport
frequent_flier
travel_date
airplane_model
frequent_flier_no
passenger_name
passenger_country
...

...
tail_no
distance
turbulence
temp_at_dept
fuel_consumed_litre
taxi_duration_mins
avg_flight_speed_kmps
engine_performance
passenger_dob
Passenger_flight_class

- Volumetrics:
 - No. of rows – 120M
 - Dataset Size – 25 GB
- Analyze the fields that are relevant to the business use cases.
- Spark UI, Memory Usage, Executor memory.

Project 3.1 – Flight Efficiency

- Identify flights with unusually high fuel consumption relative to distance travelled and average speed for their airplane model. Is there any relation between turbulence and engine performance for the models?
- Turbulence scale → 0 = no turbulence, 8 = very high turbulence. Turbulence value of 5 and above is considered as bad and impactful.
- Did distance play a role in the fuel consumption of the flights and what has high turbulence to do with engine performance and flight speed?
 - Logic for fuel economy with distance – If avg. fuel efficiency with high distance is greater than total avg. fuel efficiency then, distance had an impact of fuel economy.
 - Logic for engine performance with turbulence – If avg. engine performance with high turbulence than avg. engine performance then turbulence played a role. Note: Higher RPM is better performance.
- Convert input CSV files to a single PARQUET file.
- Spark UI – Memory Utilization, Execution Path.

Project 3.1 – Flight Efficiency – Approach

- Understand business problem statements.
 - Overall avg. fuel efficiency of an airplane model vs distance-wise avg. fuel efficiency.
 - Overall avg. engine performance vs turbulence-wise (value more than 5) engine performance.
 - Overall flight speed vs speed during turbulence.
- Pseudo-code.
 - Select relevant columns.
 - Find overall avg. fuel efficiency, engine performance and flight speed.
 - Find avg. fuel consumption, distance-wise.
 - Find avg. engine performance, turbulence-wise (when turbulence value is more than 5)
 - Find avg. flight speed, turbulence-wise.
- What transformations would be needed for above?
- How do I compare column values in the same row?
- Will persist help?

Project 3.2 – Frequent Flyer Program

- Analyze the lifetime value of frequent fliers by calculating their average total flight cost and total distance travelled, segmented by their passenger country. Identify the top 10 revenue-generating frequent fliers per country and determine the most common origin airport to destination airport routes they take.
- Hint:
 - Step 1 – Select relevant columns of frequent fliers only. (select, filter)
 - Step 2 – Find average flight cost and distance per country. (agg, groupBy)
 - Step 3 – Sum of flight cost per country and passenger and get top passengers. (sum, groupBy, orderBy)
 - Step 4 – For each source and destination, need to find the total count and then order based on count. (groupBy, count, orderBy)
- Use Spark UI to explore jobs, stages, tasks, memory (execution, storage, executor, JVM heap etc.). Experiment with input, shuffle and output partitions for better performance. Use different no. of executors with variant memory size to explore more.

Project 3.3 – Airport Operations

- For each airport, analyze the average taxi duration mins for different airplane models. Identify airports that consistently have significantly higher-than-average taxi times. Correlate this with temperature at departure. Is temperature playing any role on the taxi duration?
- Is the taxi duration for particular aircraft models different than others? indicating potential operational bottlenecks or specific ground handling issues.
- Transformation hint: `groupBy()`, `count()`, `agg()`, `selectExpr()`

Project 3.4 – Flight performance for the same route

- Identify pairs of flights of the same airplane model operating on the exact same origin airport to destination airport route, but with different tail nos. For these pairs, compare their fuel consumed per KM and avg. flight speed to identify aircraft that are consistently underperforming or overperforming for that route and model.